

Problem 1: The Last Astronaut (Pseudocode)

```
ALGORITHM LastAstronaut
INPUT N, K

INITIALIZE array/list astronot = [1, 2, ..., N]
INITIALIZE index_sekarang = 0
INITIALIZE urutan_eliminasasi = []

WHILE jumlah elemen astronot > 1 DO
    index_sekarang = (index_sekarang + K - 1) MOD jumlah elemen
    astronot
    nomor_dieliminasi = astronot[index_sekarang]

    TAMBAHKAN nomor_dieliminasi KE urutan_eliminasasi
    HAPUS astronot[index_sekarang] DARI array astronot

    IF nomor_dieliminasi MOD 2 == 0 THEN
        K = K + 2
    ELSE
        K = K - 1
    ENDIF

    IF K < 2 THEN
        K = 2
    ENDIF

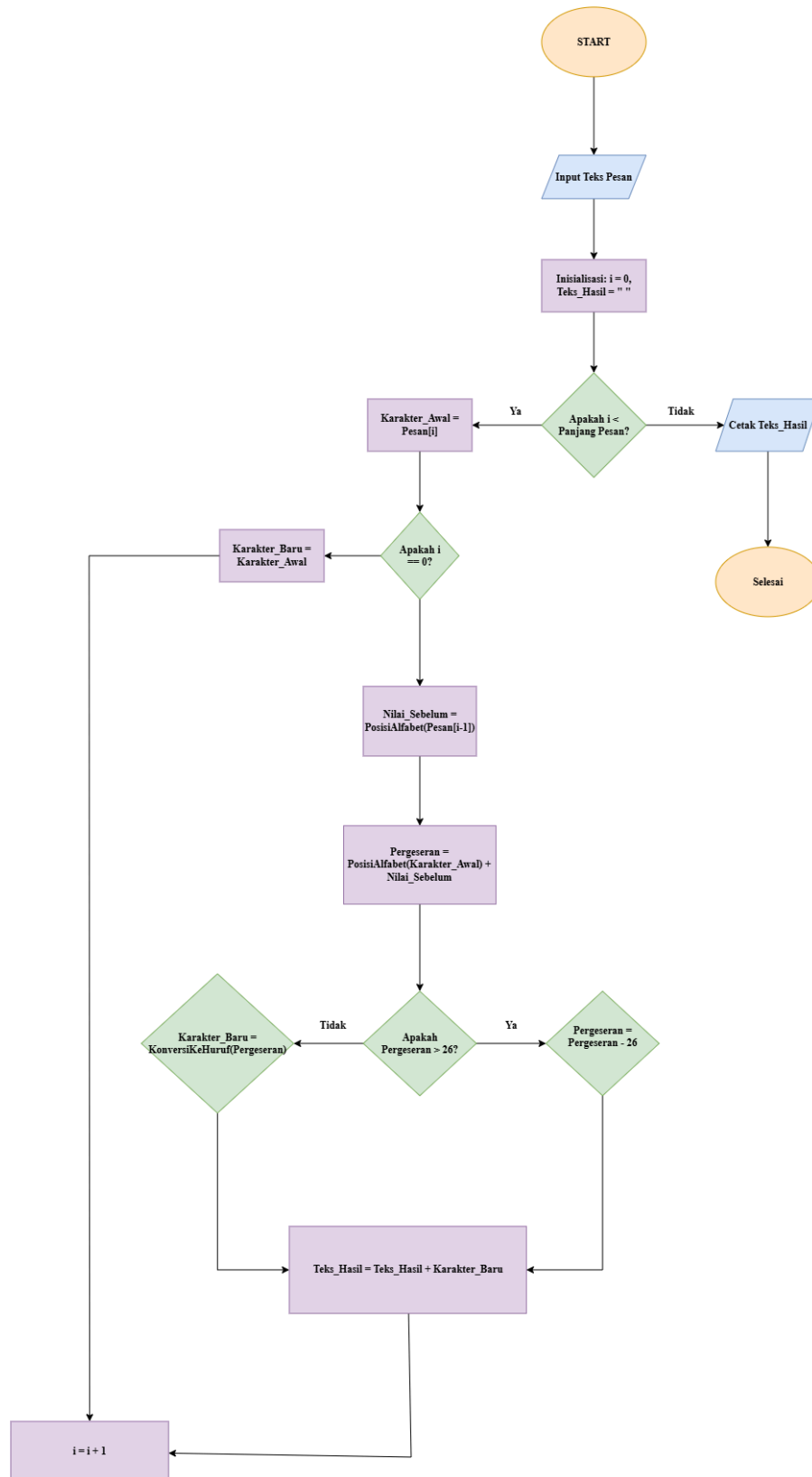
    IF index_sekarang >= jumlah elemen astronot THEN
        index_sekarang = 0
    ENDIF
ENDWHILE

OUTPUT "Urutan Eliminasi: ", urutan_eliminasasi
OUTPUT "Astronot Terakhir: ", astronot[0]
END
```

Masalah ini berpatokan pada variasi problem perulangan melingkar (Josephus Problem) dengan aturan dinamis. Logika penyelesaian dimulai dengan menyimpan seluruh astronot sebanyak N ke dalam sebuah struktur data seri/array. Simulasi dilakukan menggunakan perulangan berulang (*looping*) selama jumlah astronot tersisa lebih dari 1 orang. Posisi astronot yang tereliminasi ditentukan menggunakan perhitungan aritmatika modulo, yaitu $(\text{index_sekarang} + K - 1) \% \text{jumlah_astronot}$, sehingga penunjukan hitungan ke- K dapat mengitari lingkaran secara mulus tanpa melampaui batas indeks. Setiap kali astronot dikeluarkan dari lingkaran, nomor uniknya dicatat ke daftar eliminasi. Selanjutnya, program mengevaluasi nilai K berdasarkan kondisi ganjil atau genap dari nomor astronot tersebut. Jika nomornya

genap, K ditambah 2, sedangkan jika ganjil, K dikurangi 1. Apabila penurunan K menghasilkan nilai di bawah 2, maka K dipaksa kembali menjadi 2. Proses ini terus mengiterasi hingga tersisa 1 astronot terakhir yang dinobatkan sebagai penyintas.

Problem 2: Alien-In-The-Middle (Flowchart Text & Representation)



Sistem penyandian *Alien-In-The-Middle* mengimplementasikan enkripsi berbasis pergeseran posisi dinamis berdasarkan karakter penyusunnya. Logika dasar algoritma ini memproses setiap karakter dalam string input secara sekuensial dari kiri ke kanan. Karakter pertama pada pesan dipastikan tidak mengalami perubahan apa pun. Namun, untuk setiap karakter berikutnya (indeks $i > 1$), jumlah pergeseran ditentukan secara mutlak oleh nilai alfabetis dari karakter asli yang berada tepat di posisi sebelumnya. Proses penentuan huruf baru dilakukan dengan mengambil indeks alfabetis huruf saat ini ($A = 1, B = 2, \dots, Z = 26$) lalu menjumlahkannya dengan nilai alfabetis huruf sebelumnya. Apabila hasil penjumlahan menghasilkan nilai melebihi 26, algoritma menerapkan operasi rotasi melingkar dengan mengurangi nilai tersebut sebesar 26 agar kembali ke kisaran urutan alfabet $A-Z$. Angka posisi baru ini kemudian dikonversi kembali menjadi karakter huruf dan dirangkai ke dalam string output hingga seluruh isi pesan selesai disandikan.